integer odbc_num_rows(integer result)

The odbc_num_rows function returns the number of rows in the set, or the number of rows affected by a delete or insert if the driver supports it.

```
<?
        // connect to database
        $Connection = odbc_connect("store", "guest", "guest");

        // execute query
        $Query = "SELECT name, price ";
        $Query .= "FROM catalog ";
        $Result = odbc_do($Connection, $Query);

        print(odbc_num_rows($Result));

        odbc_close($Connection);
?>
```

integer odbc_pconnect(string dsn, string user, string password)

The odbc_pconnect function operates similarly to odbc_connect. A connection is attempted to the specified Data Source Name (DSN) and a connection identifier is returned. The connection should not be closed with odbc_close. It will persist as long as the Web server process. The next time a script executes odbc_pconnect, PHP will first check for existing connections.

```
<?
        // connect to database
        $Connection = odbc_pconnect("store", "guest", "guest");
?>
```

integer odbc_prepare(integer connection, string query)

The odbc_prepare function parses a query and prepares it for execution. A result identifier that may be passed to odbc_execute is returned. Preparing statements can be more efficient than making the driver reparse statements. This is usually the case where you have many rows to insert into the same table. To specify a value to be filled in later, use a question mark.

```
<?
        // connect to database
        $Connection = odbc_connect("store", "guest", "guest");

        // prepare query
        $Query = "INSERT INTO catalog (ID, Name, Price) ";
        $Query .= "VALUES(?, ?, ?) ";
        $Result = odbc_prepare($Connection, $Query);

        // insert
        // 0, 2000 Calendar, 20.00
        // 1, 2001 Calendar, 20.50
        // 2, 2002 Calendar, 21.00
        for($index = 2000; $index = 2002; $index++)
```